

Opiniões e experiências de professores sobre o ensino de alunos do segundo ano e Linguagens de programação subsequentes

Ethel Tshukudu
Universidade de Glasgow,
Departamento de Ciência
da Computação,
Glasgow, Reino Unido. ethel.tshukudu@glasgow.ac.uk

Quintin Cutts
Universidade de Glasgow,
Ciência da Computação,
Glasgow, Reino Unido, quintin.cutts@glasgow.ac.uk

Olivier Goletti
UCLouvain
ICTEAM/INGI
Louvain-la-Neuve, Bélgica
Universidade de Leiden
LIACS
Leiden, Países Baixos
olivier.goletti@uclouvain.be

Alaeddin da Suécia
Universidade Aberta dos Países
Baixos - Ciência
da Computação - Heerlen,
Países Baixos

Felienne Hermans
Universidade de Leiden
LIACS
Leiden, Países Baixos
ffjhermans@liacs.leidenuniv.nl

RESUMO

Motivação: Cada vez mais escolas de ensino médio estão ensinando programação e, em muitos casos, os professores ensinam várias linguagens de programação para o mesmo grupo de alunos. **Objetivos:** O objetivo deste artigo é explorar as perspectivas de professores do ensino médio sobre o ensino de uma segunda ou mais linguagens de programação, incluindo sua motivação para ensinar múltiplas linguagens, suas dificuldades e o uso de estratégias de transferência ao ensinarem sua segunda ou terceira linguagem de programação. **Método:** O estudo consiste em entrevistas semiestruturadas com 23 professores do ensino médio em dois países europeus. **Resultados:** Nossos resultados indicam que os alunos do ensino médio enfrentam os mesmos problemas que os universitários ao passarem da primeira para as seguintes linguagens. Além disso, as atitudes dos professores em relação ao aprendizado de uma segunda língua são bastante variáveis, tanto positivas quanto negativas, com algumas estratégias de ensino de apoio sendo utilizadas, mas também com muitas outras menos eficazes. **Discussão:** Nossos resultados sugerem que o valor do aprendizado de uma segunda língua precisa ser mais enfatizado nos materiais de desenvolvimento profissional de professores e que os professores podem precisar de mais apoio na implementação de estratégias de transferência.

CONCEITOS CCS

• Temas sociais e profissionais y Educação em ciência da computação.

PALAVRAS-CHAVE

linguagens de programação; transferência; compreensão de código; desenvolvimento conceitual; sintaxe; semântica; K12

Formato de referência da ACM:

Ethel Tshukudu, Quintin Cutts, Olivier Goletti, Alaeddin Swidan e Felienne Hermans. 2021. Opiniões e experiências de professores sobre o ensino de uma segunda e subsequentes linguagens de programação. Em Anais da 17ª Conferência Internacional da ACM sobre Pesquisa em Educação em Computação (ICER 2021), 16 a 19 de agosto de 2021, Evento Virtual, EUA. ACM, Nova York, NY, EUA, 12 páginas. <https://doi.org/10.1145/3446871.3469752>

1 INTRODUÇÃO

Os programas de Ciência da Computação (CC) nas universidades tendem a envolver mais de uma linguagem de programação (LP). As sequências universitárias geralmente incluem alguma combinação de linguagens orientadas a objetos (por exemplo, Java), de nível de sistema (por exemplo, C) e funcionais (por exemplo, Scheme), bem como linguagens específicas de domínio, por exemplo, bancos de dados e web, como SQL e HTML. Agora que a programação também é cada vez mais ensinada no ensino médio, os currículos escolares nacionais (por exemplo, o K-12 CS Framework [44] dos EUA e o currículo de CC inglês para escolas [6]) também incluem a expectativa de ensinar várias linguagens de programação.

A aprendizagem de uma segunda ou subsequente linguagem de programação tem sido objeto de estudo há muitas décadas. Enquanto trabalhos anteriores se concentraram nas questões enfrentadas por programadores profissionais ao trocar de linguagem de programação [32–35, 50], pesquisas mais recentes têm se concentrado em programadores quase iniciantes e sua progressão da primeira para a segunda linguagem [2, 8, 18, 19, 43, 47]. Pesquisas sobre transferência de linguagem de programação relataram que a transferência conceitual da primeira para a linguagem de programação subsequente muitas vezes não ocorre como esperado em sala de aula. Os alunos frequentemente demonstram pouca capacidade de aplicar o conhecimento adquirido em linguagens anteriores ensinadas em sala de aula a novas linguagens, e isso é especialmente verdadeiro para conceitos abstratos [2, 31, 41, 42]. Além disso, pesquisas relatam que a transferência negativa da linguagem anterior pode dificultar a aprendizagem de uma nova linguagem [33, 45, 47]. Apesar o

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou em sala de aula, sem custo, desde que as cópias não sejam feitas ou distribuídas para fins lucrativos ou comerciais e que as cópias contenham este aviso e a citação completa na primeira página. Os direitos autorais dos componentes desta obra que pertencem a terceiros que não a ACM devem ser respeitados. É permitido fazer resumos com a devida atribuição. Para copiar, republicar, postar em servidores ou redistribuir para listas, é necessária permissão específica prévia e/ou o pagamento de uma taxa. Solicite permissões pelo e-mail permissions@acm.org. ICER 2021, 16 a 19 de agosto de 2021, Evento Virtual, EUA © 2021 Association for Computing Machinery. ACM ISBN 978-1-4503-8326-4/21/08...\$15,00 <https://doi.org/10.1145/3446871.3469752>

ICER 2021, 16 a 19 de agosto de 2021, Evento Virtual, EUA

Foram relatadas falhas na transferência, mas também alguns sucessos. também [8, 35, 41]. Estes geralmente ocorrem quando línguas anteriores compartilham Semelhanças sintáticas e semânticas com novas línguas.

Para fortalecer a transferência, trabalhos anteriores examinaram o ensino para transferência em programação no ensino superior [31, 43]. Estes Estudos demonstraram que, quando os educadores se concentram especificamente na transferência de conhecimento em sala de aula, isso pode melhorar a aprendizagem dos alunos. Portanto, A compreensão das estratégias de transferência por parte dos professores é crucial para a qualidade do ensino da computação. Explorar suas perspectivas e métodos de... As estratégias de transferência podem nos permitir identificar necessidades específicas para os professores. educação. Ao contrário de trabalhos anteriores, que exploraram principalmente o ensino para Na transferência para instituições de ensino superior, focamos no ensino secundário. educação (K-12).

Portanto, o objetivo deste artigo é relatar as práticas e atitudes atuais dos professores do ensino médio em relação ao ensino de segunda língua. aprendizagem e a consciência dos professores, a partir de suas próprias experiências em sala de aula, sobre as questões de transferência. Para responder às nossas quatro questões de pesquisa, vinte e três professores do ensino fundamental e médio, cada um de uma escola diferente em dois Foram entrevistados países europeus.

Como ponto de partida, focamos nos motivos para o ensino de múltiplas línguas. línguas. A abordagem que um professor adota para ensinar uma segunda língua será significativamente moldada por suas crenças sobre o valor e objetivo do ensino da segunda língua, e por isso nossa primeira pesquisa A questão é:

RQ1: Por que os professores ensinam uma segunda linguagem de programação?

Não está claro se os alunos do ensino fundamental e médio enfrentam os mesmos problemas de transferência ao aprenderem sua segunda linguagem de programação. estudantes universitários. Portanto, nossa segunda questão de pesquisa é:

RQ2: Que tipos de benefícios/problemas os professores notam quando Ensinar uma segunda ou subsequente linguagem de programação?

Também exploramos como as linguagens de programação subsequentes são ensinado, com ênfase especial nas estratégias de transferência entre línguas. Pode ser que os professores do ensino médio, que normalmente recebem um treinamento mais extenso do que os professores universitários, sejam já conhecem e utilizam métodos de ensino que levam em consideração das questões positivas e negativas associadas à transferência. Portanto, Nossas questões finais de pesquisa são:

RQ3: Quais são as opiniões dos professores de informática sobre o uso de estratégias de transferência?

RQ4: Que tipos de estratégias de transferência os professores de informática utilizam? usar?

A análise das 23 entrevistas com professores corrobora o seguinte: Resultados: os professores têm uma variedade de motivações para ensinar mais. mais de uma linguagem de programação, algumas das quais provavelmente não serão utilizadas. estão bem alinhadas com a aprendizagem por transferência bem-sucedida; as questões de transferência são com experiência em salas de aula, assim como em universidades; professores A maioria está ciente desses problemas de transferência até certo ponto, mas... pouco para utilizar ativamente esse conhecimento em sua abordagem a Ensino de uma segunda língua; alguns professores utilizam o ensino de apoio. estratégias, mas muitas estratégias menos úteis também são usadas.

Ethel Tshukudu, Quintin Cutts, Olivier Goletti, Alaaeddin Swidan e Felienne Hermans

2. CONTEXTO

Nesta seção, discutimos trabalhos anteriores sobre os desafios que os alunos enfrentam. Abordamos os desafios enfrentados ao aprender a programar. Também exploramos a transferência entre linguagens de programação e estratégias de ensino existentes. favorecendo a transferência.

2.1 Pesquisa sobre diferentes tipos de programação línguas

Os estudantes de informática se deparam constantemente com a necessidade de navegar entre diferentes linguagens de programação durante sua formação. Muitos departamentos de Ciência da Computação (CC) de universidades ensinam aos alunos várias linguagens de programação em cursos introdutórios de programação. Por exemplo, Davies et al. [9] relataram em todo o país Resultados de uma pesquisa realizada com 371 departamentos de Ciência da Computação de graduação nos EUA sobre línguas e métodos que eles usam em um curso introdutório de três disciplinas sequência (CS0-CS1-CS2).¹ Este estudo relatou que as linguagens mais comuns usadas em um curso CS0 foram Alice, Python e Visual Studio. Básico e JavaScript, sendo Alice a mais popular. Após concluir o CS0, os alunos geralmente precisam cursar CS1 e/ou CS2.

que ensinam principalmente programação com diferentes abordagens orientadas a objetos. linguagens como Java e C++.[9]. Ensino de uma linguagem de programação visual como Scratch no CS0 antes da transição para Java em CS1 também é relatado em outros estudos [20, 49]. Em alguns casos, instituições adotaram uma linguagem baseada em texto, como Python, antes transição para Java [22].

Abordagens semelhantes são adotadas em algumas escolas primárias e secundárias. escolas. Por exemplo, no currículo de informática do Reino Unido, os requisitos para alunos com menos de 11 anos podem ser totalmente atendidos no Scratch. No entanto, quando Os alunos têm entre 11 e 14 anos, e o currículo recomenda que Eles devem usar duas ou mais linguagens de programação, pelo menos uma. dos quais é textual, para resolver uma variedade de problemas computacionais [6].

2.2 Problemas de sintaxe na programação Educação

Muitos aspectos da aprendizagem da primeira linguagem de programação são desafiadores. O primeiro problema com que os aprendizes têm dificuldade é a sintaxe. linguagens de programação. A precisão necessária durante a programação é frequentemente um fator que contribui para a dificuldade de aprendizado. programação. Requer “um nível de atenção aos detalhes que não vêm naturalmente aos seres humanos” [24]. Denny et al. descobriram que fraco Em 73% dos casos, os alunos entregam código-fonte com erros de sintaxe. e, mesmo os melhores alunos o fazem em 50% dos casos [10]. Altadmri e Brown [1] analisou 37 milhões de compilações de 250.000 estudantes e O erro mais comum identificado foi o erro de sintaxe: incompatibilidade. colchetes, que ocorreram em quase 800.000 compilações. Outros Pesquisadores descobriram que linguagens de programação comuns como Java e Perl são mais difíceis de entender do que uma linguagem aleatória, enfatizando as dificuldades que os novatos enfrentam na compreensão da sintaxe [37]. Quando se trata de aprender uma segunda linguagem de programação, a sintaxe continua sendo um problema. Estudos mostraram que, durante os estágios iniciais, Ao aprender uma nova linguagem, seja de programação [3] ou natural [15], os alunos fazem correspondências sintáticas entre a linguagem anterior e a nova linguagem. Tentativas de abordar essas transições sintáticas.

¹Neste contexto, CS0 é o curso introdutório sem pré-requisitos, enquanto CS1 e CS2 são o primeiro e o segundo cursos obrigatórios da sequência, respectivamente.

problemas resultaram em ferramentas para dar suporte a interfaces com linguagem visual e textual correspondente para que os alunos formem analogias no nível da sintaxe à medida que fazem a transição [8, 13, 18, 46]. Curiosamente, os alunos avaliam o aprendizado da sintaxe como mais problemático do que os professores [29], o que pode nos ajudar a entender por que pouco esforço é dado à explicação explícita da sintaxe em muitas salas de aula de programação.

2.3 Pesquisa sobre transferência entre linguagens

de programação. Pesquisas sugerem

que ambientes de programação baseados em blocos, embora bem-sucedidos em mudar atitudes e motivar os alunos [2], podem não prepará-los suficientemente para a transição para linguagens de programação baseadas em texto [25, 30, 47]. No entanto, um estudo de Weintrop e Wilensky [47] relatou que ambientes de programação baseados em blocos podem ser úteis se o professor realmente ensinar com a transferência em mente. Powers et al. [30] relataram problemas de transição após observarem alunos que passaram de 7 semanas de aprendizado de Alice para Java em um semestre. Eles relataram que os alunos tiveram dificuldades com a sintaxe de Java e alguns não conseguiram mapear conceitos como declarações de objetos, variáveis de estado e métodos em Java com os conceitos que haviam aprendido em Alice. Outro estudo [25] com resultados semelhantes relatou que alunos que tinham conhecimento de Pascal e estavam aprendendo Alice não conseguiam fazer a conexão lógica dos conceitos de fluxo de controle (por exemplo, loops) entre essas duas linguagens sem instrução explícita. Weintrop e Wilensky [47] também relataram que os alunos tiveram dificuldades com erros de Java, como variáveis indefinidas e tipos incompatíveis, ao fazer a transição de linguagens baseadas em blocos para Java.

Outros estudos relataram problemas de transição que os alunos frequentemente enfrentam ao transitar entre diferentes linguagens textuais [31, 41, 45, 48]. Por exemplo, em seu experimento com alunos migrando de Pascal para uma nova linguagem Ada, Walker e Schach [45] relataram que alguns alunos preferiram escrever programas em Ada usando uma estrutura de laço while familiar de sua linguagem anterior, em vez de mudar para a estrutura loop-exit-end do Ada. Isso significa que eles não aproveitaram os novos recursos oferecidos pela nova linguagem. Outro estudo [31] relatou que os programas desenvolvidos por alunos em uma nova linguagem (Java orientada a objetos) foram influenciados pelo conhecimento de sua linguagem anterior (Racket, linguagem funcional). Outros estudos relataram transferência de planos quando programadores experientes resolviam problemas em uma nova linguagem [33–35].

Apesar dos efeitos negativos da transferência relatados, os pesquisadores também relataram alguns efeitos positivos. A transferência bem-sucedida ocorre especialmente quando a linguagem anterior e a nova linguagem são semelhantes. Por exemplo, Scholtz e Wiedenbeck [33] relataram que foi mais fácil para os alunos aprenderem as estruturas de repetição na nova linguagem Icon porque elas pareciam e funcionavam de maneira semelhante às estruturas de repetição na linguagem Pascal que eles aprenderam antes do Icon.

Um esforço para ajudar a compreender melhor o processo de transferência é relatado no recente Modelo de Transferência de Linguagem de Programação (MPLT) validado por Tshukudu e Cutts [42]. O MPLT descreve o processo de transição e como a aprendizagem de conceitos de programação por programadores iniciantes é afetada durante a mudança [42]. No modelo,

Os aprendizes realizam automaticamente uma transferência semântica entre linguagens com base em similaridades sintáticas, o que pode resultar em transferência tanto positiva quanto negativa. Além disso, sugerem que a transição para novas linguagens pode, por vezes, não ser tão automática quanto os educadores esperam, e isso pode representar um desafio para programadores relativamente iniciantes. Acreditamos que o ensino pode ser planejado para promover a transferência da aprendizagem.

2.4 Estratégias de transferência conhecidas

Thorndike e Woodworth [40] estiveram entre os primeiros a avaliar a transferência da aprendizagem por meio de testes formais e propuseram que a aprendizagem não se tratava de uma habilidade geral. Como a transferência envolve conhecimento prévio, reforçar a aprendizagem inicial é o primeiro passo a ser considerado. Identificar e ativar ativamente o conhecimento prévio pode ajudar na transferência, em vez de considerá-la um processo passivo. Por exemplo, Klahr e Carver avaliaram a aprendizagem inicial do LOGO e a relacionaram à transferência para tarefas correlatas [17]. Além disso, o contexto importa, e a transferência da aprendizagem é aprimorada quando os conceitos são ensinados em múltiplos contextos, em vez de exemplos excessivamente contextualizados [11]. Estratégias de autorregulação podem auxiliar os alunos no processo de aprendizagem e transferência de conhecimento [4]. A transferência da aprendizagem também se beneficia de uma compreensão mais abstrata dos conceitos [17]. Isso, de fato, demonstra um estágio mais avançado de aprendizagem na taxonomia SOLO [5], também frequentemente usada para avaliar os níveis de compreensão de iniciantes em ciência da computação. O desenvolvimento dessa compreensão mais abstrata pode ser feito destacando os principais subobjetivos no material instrucional [21] ou comparando e contrastando exemplos resolvidos [26]. Goletti et al. [12] também sugerem que o uso da prática instrucional explícita favorece a transferência.

Perkins e Salomon [28] propõem duas técnicas específicas para promover a transferência positiva da aprendizagem, respectivamente, Hugging e Bridging: Hugging visa aumentar as semelhanças entre os domínios aprendidos e o domínio alvo, enquanto Bridging promove associações conceituais mais abstratas entre a aprendizagem inicial e seu domínio alvo. Trabalhos anteriores que adotaram essas técnicas de transferência estão principalmente no contexto da transferência de linguagens baseadas em blocos para linguagens baseadas em texto [8, 18, 23, 39, 47]. Por exemplo, em seu experimento de transição de alunos de Alice para Python, Tabet et al. [39] usaram as técnicas de bridging para transferir conceitos de Alice para Python. Essa intervenção foi realizada mostrando explicitamente exemplos semelhantes em Alice e Python (por exemplo, loops, instruções if, métodos) para ajudar os alunos a transferir conceitos de forma eficaz. Os resultados mostraram que os alunos que participaram da intervenção de transferência encontraram

foi mais fácil para aqueles que fizeram a transição de Alice para Python do que para aqueles que não fizeram. Outro estudo de Dann et al. [8] utilizou a técnica de ponte usando um ambiente híbrido que permitia aos alunos alternar entre blocos e notação de texto ao fazer a transição de Alice para Java. Esses estudos mostram que as estratégias de transferência podem ser bem-sucedidas em ajudar os alunos na transferência, porém não abordam a transferência negativa, que tem sido relatada como persistente na aprendizagem de uma segunda língua [42].

Para atender à transferência positiva e negativa, Tshukudu e Jensen [43] realizaram uma intervenção de transferência alinhada ao MPLT. O estudo empírico teve como objetivo investigar a eficácia do ensino explícito no ensino de conceitos de uma segunda linguagem de programação. O estudo foi realizado com estudantes de graduação que estavam em processo de transição.

Os alunos foram orientados a migrar de Python para Java. Foi permitido comparar e contrastar as semelhanças e diferenças semânticas entre fragmentos de código equivalentes em Java (nova linguagem) e Python (linguagem já conhecida), e quaisquer equívocos foram explicitamente corrigidos. Os resultados demonstraram benefícios na transferência conceitual e na correção de equívocos causados pela transferência semântica negativa.

Não temos conhecimento de pesquisas que investiguem as experiências, atitudes e abordagens dos professores da rede pública em relação ao ensino de uma segunda língua; portanto, tomamos isso como ponto de partida.

3. CONFIGURAÇÃO DA PESQUISA

O objetivo deste artigo é **compreender as perspectivas e práticas de professores de informática sobre o uso de estratégias de transferência no ensino de uma segunda e de linguagens de programação subsequentes**. Para tanto, realizamos entrevistas com 23 professores em exercício, 13 em inglês e 10 em holandeses.

3.1 Participantes

Vinte e três professores de informática do ensino fundamental e médio, cada um de uma escola diferente em dois países europeus, participaram do estudo: participantes T1-T23. A participação no estudo foi voluntária e os professores tiveram a garantia de confidencialidade e anonimato de suas respostas antes das entrevistas. Recrutamos os professores por meio de múltiplos canais. Na Escócia, contatamos diretamente uma lista de professores interessados em pesquisa em Ciência da Computação. Na Holanda, publicamos um anúncio no boletim informativo semanal da *Wetenschap en Technologie*: uma associação de professores de Ciência da Computação do ensino fundamental e médio na Holanda. Além disso, os participantes foram contatados por meio de uma conta pública no Twitter seguida por muitos professores. Como resultado, os participantes deste estudo estão posicionados no extremo do espectro em que os professores estão ativamente engajados em atividades de pesquisa e participando de conferências que promovem novas ideias e atividades no ensino de Ciência da Computação.

A Tabela 1 apresenta uma visão geral dos dados demográficos e informações relacionadas ao ensino dos professores. Sua experiência no ensino de Informática varia de 5 a 35 anos. Todos os participantes lecionam em escolas públicas de ensino fundamental II/médio, enquanto dois professores têm carga horária adicional relacionada ao último ano do ensino fundamental I. Os professores que entrevistamos lecionam para alunos com idades entre 11 e 18 anos. Em termos do currículo seguido pelos professores, existem diferenças entre os dois países. Na Escócia, existe um currículo nacional de Ciência da Computação, no qual os alunos dos dois primeiros anos do ensino secundário (S1 e S2) geralmente têm uma aula de 50 minutos por semana, embora essa duração possa variar, e não há exame obrigatório. A partir do terceiro ano do ensino secundário (S3), os alunos que optam pela disciplina de Ciência da Computação passam a ter entre duas e quatro aulas de 50 minutos por semana, além de realizarem exames. Nos Países Baixos, não existe um currículo nacional, mas sim diretrizes sobre o que ensinar em alfabetização digital e Ciência da Computação no ensino secundário, divididas em domínios ou temas. Os alunos não fazem exames obrigatórios ao seguirem esses domínios ao longo dos anos escolares.

Como resultado, os professores na Holanda têm mais flexibilidade e responsabilidade para decidir o que e como ensinar quando se trata de Ciência da Computação e como avaliar o aprendizado de seus alunos.

[2https://ieni.org/](https://ieni.org/)

3.2 Protocolo de entrevista

As entrevistas semiestruturadas foram conduzidas pelos autores, sendo as perguntas previamente elaboradas complementadas por perguntas de acompanhamento, sondagens e comentários quando necessário.

Para responder à RQ1 sobre a motivação dos professores para ensinar uma segunda linguagem de programação, pedimos que justificassem a escolha da linguagem e os objetivos de aprendizagem que buscavam alcançar ao ensiná-la. Para responder à RQ2 sobre os problemas e benefícios de ensinar mais de uma linguagem, questionamos se observavam vantagens ou desvantagens ao ensinar uma linguagem subsequente e se isso se relacionava com a primeira ou com a linguagem anterior. Para responder à RQ3 sobre suas opiniões a respeito de estratégias de transferência, perguntamos sobre o impacto percebido das linguagens ensinadas anteriormente na aprendizagem da segunda/subsequente linguagem de programação pelos alunos e se consideravam benéfico implementar estratégias de transferência. Para responder à RQ4 sobre as estratégias de transferência que utilizam em seu ensino, pedimos que descrevessem seus métodos de ensino da segunda linguagem de programação; pontos específicos de atenção foram como se referiam à primeira ou à(s) linguagem(ns) anterior(es).

Para coletar dados abertos sobre os pensamentos e crenças dos professores a respeito da transferência e do ensino de uma segunda língua e de línguas de programação subsequentes, foram utilizadas entrevistas semiestruturadas. As entrevistas com cada professor duraram, em média, 50 minutos. Todas as entrevistas foram realizadas por videochamada, gravadas, transcritas e, em seguida, codificadas. Todos os professores assinaram um termo de consentimento livre e esclarecido. No termo, apresentamos brevemente o objetivo da pesquisa e garantimos que todos os dados obtidos nas entrevistas seriam anonimizados.

3.3 Processo de codificação

A análise temática [7] foi utilizada para analisar e interpretar os temas presentes nos dados. Importamos as transcrições das entrevistas para o Excel para uma análise de codificação detalhada. O processo de codificação foi dividido em três fases distintas. Primeiramente, cada uma das quatro questões de pesquisa foi codificada individualmente por um dos quatro autores. Em seguida, o autor discutiu os códigos iniciais com outro autor, e os conflitos foram debatidos e resolvidos. Temas menores foram agrupados e temas maiores foram divididos por meio de um processo iterativo. Nesse processo, buscamos encontrar temas que descrevessem as questões centrais de cada questão de pesquisa. Nosso objetivo era ter cerca de três temas principais por questão de pesquisa, possivelmente com subtemas. As discussões entre o primeiro e o segundo codificadores giraram em torno do agrupamento preciso de temas e subtemas. Uma discussão mais aprofundada abordou a inclusão ou exclusão de algumas citações nos subtemas.

Por fim, todos os códigos e suas implicações foram discutidos com os dois autores restantes, que não participaram do processo inicial de codificação nem da discussão subsequente. Nesta etapa final de consolidação, os códigos também foram comparados entre as questões de pesquisa, quando relevante, para evitar sobreposições entre elas e identificar temas genéricos para discussão aprofundada na seção de discussão.

4 RESULTADOS

Os dados revelaram diversos temas, que serão apresentados em quatro subseções correspondentes às questões de pesquisa.

Código	Gênero	País	Nível escolar (idade dos alunos)	Ensino experiment- ência	Linguagens de programação ensinadas
T1	Macho	Escócia	Ensino Médio (12-18 anos)	21 anos	Editor de blocos Scratch, Blockly e micro:bit , HTML, Python
T2	Macho	Escócia	Ensino Médio (12-18 anos)	10 anos	Scratch, editor de blocos Scratch e micro:bit , HTML, Python, PHP
T3	Macho	Escócia	Ensino Médio (12-18 anos)	18 anos	Scratch, HTML, Python, PHP
T4	Ensino secundário/médio feminino na Holanda (15-17 anos)			5 anos	Python, HTML/CSS, JavaScript, C#
T5	Macho	Ensino secundário/médio na Holanda (15-16 anos)		5 anos	HTML, Scratch, Javascript
T6	Macho	Ensino secundário/médio na Holanda (15-17 anos)		5 anos	Processamento (Java), Arduino, Web (PHP), Python
T7	Macho	Ensino secundário/médio na Holanda (16-18 anos)		11 anos	HTML, Python, Java
T8	Escócia feminina		Ensino Médio (12-18 anos)	15 anos	Blockly, Scratch e HTML, Python e SQL, Python, HTML/CSS Javascript, MySQL e PHP
T9	Escócia feminina		Ensino Médio (12-18 anos)	30 anos	Blockly, Scratch, Python
T10	Macho	Escócia	Ensino Médio (12-18 anos)	35 anos	Blockly, Scratch, Java, HTML
T11	Macho	Escócia	Ensino Médio (12-18 anos)	16 anos	Blockly, Scratch, Javascript e VB, JavaScript
T12	Escócia feminina		Ensino fundamental (8-11 anos) Ensino Médio (12-18 anos)	19 anos	Scratch, editor de blocos micro:bit, Turtle Python, Python
T13	Macho	Ensino secundário/médio na Holanda (16-18 anos)		35 anos	HTML, JavaScript, PHP, Python
T14	Masculino	Escócia	Ensino Médio (12-18 anos)	24 anos	Scratch(S1), HTML/CSS SQL, VB, JavaScript, PHP MySQL
T15	Macho	Escócia	Ensino Médio (12-18 anos)	15 anos	Scratch, Python
T16	Macho	Escócia	Ensino Médio (12-18 anos)	23 anos	Truebasic, Scratch, VB, Python, Java
T17	Ensino secundário/médio feminino na Holanda (16-18 anos)			14 anos	HTML/CSS, Fluxogramas, Flowgorithm Python
T18	Macho	Ensino secundário/médio na Holanda (16-18 anos)		3 anos	Python, Banco de Dados SQL, JavaScript
T19	Macho	Ensino secundário/médio na Holanda (13-18 anos)		14 anos	Scratch, Python
T20	Macho	Ensino secundário/médio na Holanda (16-18 anos)		5 anos	HTML/CSS, JavaScript, AJAX e JSON, PHP MySQL, Python
T21	Macho	Ensino secundário/médio na Holanda (16-18 anos)		32 anos	Python, PHP
T22	Escócia feminina		Ensino fundamental (11-12 anos) Ensino Médio (12-18 anos)	23 anos	Editor de blocos Micro:bit, Scratch, Python, Javascript SQL HTML/CSS, PHP
T23	Macho	Escócia	Ensino Médio (12-18 anos)	15 anos	Scratch, Python

Tabela 1: Detalhes sobre os professores participantes. O código do professor reflete a ordem de sua entrevista, onde T1 é o primeiro entrevistado e T23 é o último entrevistado.

4.1 RQ1: Razões para o uso de vários idiomas

Ao analisar as razões apresentadas por professores do ensino médio para o ensino de vários idiomas, encontramos três razões principais: idiomas fazem parte do currículo ou existe material didático específico para cada tema. línguas (7 professores), começando com uma língua simples (13 professores), e diferentes línguas têm impactos diferentes nos alunos (10 professores). As duas últimas categorias podem ser subdivididas, como Explicaremos isso no restante desta seção.

4.1.1 Parte do currículo ou dos materiais. Primeiro, veremos as razões pelas quais Os professores ensinam vários idiomas, que não estão diretamente relacionados. a linguagens de programação específicas ou práticas de ensino. Sete Os professores mencionaram razões como, por exemplo, um currículo fixo, sendo este o motivo.

principal motivo para ensinar diferentes idiomas ou o fato de que os materiais estavam disponíveis para idiomas específicos.

4.1.2 Comece com uma linguagem simples. A maior categoria que nós Entre os 23 professores entrevistados, mencionados treze vezes, está que os professores querem começar com uma linguagem simples. Frequentemente, mas não. Essa linguagem simples é sempre uma linguagem baseada em blocos. Professores Explicou que, em determinado momento, as línguas mais simples deixam de ser suficientes, seja porque as crianças as superaram ou porque não oferecem o suficiente. possibilidades de ensino. Nesse ponto, surge a necessidade de um segunda ou terceira língua. Distinguimos quatro subcategorias na categoria abrangente de "Começando com uma linguagem simples".

A linguagem simples gera engajamento. O primeiro motivo que observamos, mencionado por oito professores, é que uma linguagem simples ajuda

Categoria	Número de professores
Parte do currículo	7
Comece com uma linguagem simples.	13
- Linguagem simples proporciona engajamento	8
- Linguagem simples evita erros	5
- A linguagem simples gera confiança.	4
- A linguagem simples facilita o foco nos conceitos.	3
Diferentes idiomas oferecem diferentes benefícios.	10
- Diferentes idiomas têm diferentes aplicações	4
- Conhecer diferentes idiomas proporciona uma perspectiva diferente. 6	

Tabela 2: Razões apresentadas pelos professores para o ensino de vários idiomas

para que os alunos se envolvam rapidamente com a programação. Por exemplo, T1 afirma que "Começamos com uma linguagem simples, para não os assustar". T8 acrescenta que "[O Scratch] é mais fácil e [usa] blocos, e os alunos podem inserir algo rapidamente".

A linguagem simples gera confiança. Quatro professores mencionam o fato de que uma linguagem simples aumenta a autoconfiança dos alunos, elevando as chances de que eles gostem de programar. Por exemplo, o professor T16 afirma que eles "começam com blocos para evitar frustração".

Isso parece estar relacionado ao fato de que, como sabemos por pesquisas anteriores descritas na Seção 2, a sintaxe pode ser uma grande barreira para os alunos. Começar com uma linguagem simples, geralmente baseada em blocos, ajuda a adiar a barreira da sintaxe para um momento posterior do curso, quando os alunos podem se sentir mais preparados para a programação.

Linguagem simples evita erros. Cinco professores relatam que começam especificamente com uma linguagem simples porque isso facilita para os alunos evitarem erros de sintaxe. Por exemplo, o professor T9 afirma: "Começamos com o Scratch para eliminar erros de sintaxe (S1-S2)". ...

Ensinar várias linguagens de programação é visto como uma oportunidade para ensinar os alunos a pensar sobre linguagens de programação e suas diferenças. Por exemplo, T5 explica que "explicamos que com o Scratch não é possível criar um site, o que faz com que o aluno queira aprender uma nova linguagem". Evitar erros pode ser relacionado à nossa categoria anterior de desenvolvimento da confiança.

Uma linguagem simples facilita o foco nos conceitos. O último motivo apresentado por três professores é a crença de que uma linguagem simples torna mais fácil para os alunos se concentrarem nos conceitos de programação em vez de em questões sintáticas. Por exemplo, T17 afirma: "Usamos fluxogramas executáveis porque eles ajudam a explicar os três conceitos básicos: sequência, seleção e iteração. Depois, 'aprimoramos' o código com uma linguagem de programação textual."

4.1.3 Diferentes línguas oferecem diferentes benefícios. A terceira categoria principal é que o uso de diferentes línguas traz diferentes benefícios educacionais para os alunos. Dez professores citam isso como um motivo para

Utilizando várias linguagens. Por exemplo, T20 afirma: "Eu uso linguagens diferentes especificamente para objetivos diferentes. Primeiro HTML/CSS para mostrar a eles que o código está em toda parte". Dividimos essa categoria principal em duas subcategorias.

Diferentes idiomas têm diferentes aplicações. Uma das razões pelas quais pode ser benéfico ensinar vários idiomas é chamada de linguagem.

O objetivo desta aula, ministrada por quatro professores, é mostrar aos alunos que diferentes linguagens de programação têm diferentes aplicações. Se você quer que os alunos percebam a abrangência da programação, isso pode ser feito através de diversas linguagens. T6 explica: "Diferentes linguagens de programação ajudam a ensinar diferentes partes da programação. Por exemplo, C++ é difícil, mas ajuda a ensinar hardware."

Conhecer diferentes idiomas proporciona uma perspectiva diferente. Por fim, seis professores mencionaram o fato de que o conhecimento de vários idiomas por parte dos alunos lhes dará uma melhor compreensão das linguagens de programação. Por exemplo, a T21 espera que "os alunos aprendam um novo idioma mais rapidamente se já tiverem tido contato com dois [idiomas]".

4.2 RQ2: Problemas/benefícios do ensino de múltiplas linguagens. Os

professores foram solicitados a

refletir sobre os benefícios e problemas que observam nos alunos quando estes migram para outra linguagem de programação. Categorizamos suas respostas em uma lista de benefícios e outra de problemas. A Tabela 3 apresenta um resumo das categorias de benefícios e problemas.

4.2.1 Benefícios observados ao migrar para outra linguagem de programação. Categorizamos os benefícios em duas categorias principais: efeitos positivos na compreensão de conceitos de programação em PL2 (15 professores) e efeitos positivos nas habilidades cognitivas (7 professores).

Efeito positivo na compreensão de conceitos de programação em PL2. Esta categoria é a maior entre os benefícios, destacada por 15 professores diferentes. Distinguimos aqui três subcategorias. Primeiro, os alunos transferem a compreensão de conceitos básicos de programação de uma linguagem para outra. Por exemplo, T4 afirma que "os alunos entendem como usar instruções if em VB porque aprenderam isso no Scratch, o que acontece de forma bastante natural". Da mesma forma, T9 destaca que os alunos "transferem automaticamente o conceito de instruções if e repetições do Scratch". Segundo, os benefícios da compreensão conceitual decorrem da conexão explícita que os alunos estabelecem com a linguagem de programação anterior para ajudar a resolver problemas na nova linguagem. Essa conexão pode surgir apenas da memorização do conceito da primeira linguagem ou da repetição do conceito pelos professores. Por exemplo, T4 destaca que "às vezes os próprios alunos dizem: 'em Python, você faria isso assim ou assado, mas em C# eu faço diferente agora'". De acordo com alguns professores, essa conexão pode levar os alunos a aprenderem PL2 mais rapidamente, como T6 coloca: "repetição de conceitos, [e então] você tem a base para escolher

Categoria de benefícios	Número de professores
Efeito positivo na compreensão de conceitos de programação em PL2	15
- Transferir a compreensão de conceitos básicos de programação de uma linguagem para outra. 11	7
- Os alunos estabelecem a conexão/ligação com o idioma anterior.	2
- Ocorre apenas sob condições específicas.	
Efeito positivo nas habilidades cognitivas	7
- Transferência de habilidades de resolução de problemas e pensamento abstrato	4
- Ampliar as perspectivas dos alunos	3
Categoria do problema	
Efeito negativo da transferência sintática	8
A troca de contexto é difícil e demorada.	5
- Mistura de elementos sintáticos de duas línguas	5
Efeito negativo da transferência semântica	10
- Conceitos errôneos: Alunos fazendo suposições sobre semântica	6
- Dificuldade com a semântica	6
Os alunos não estabelecem nenhuma conexão com o idioma anterior.	4
Alunos desmotivados ou com pouca confiança para aprender um novo idioma.	3
- Alunos que generalizam a falta de habilidade com base em uma experiência negativa anterior.	3
- Presos a um único idioma que conhecem	1

Tabela 3: Benefícios e problemas destacados por professores no ensino de múltiplas línguas

Aprender outro idioma rapidamente". Terceiro, alguns professores enfatizam que para Qualquer benefício decorrente da compreensão de que isso ocorra, condições específicas devem aplicar, como “boas instruções, discussão entre os alunos e foco em questões de compreensão de código”, conforme destacado por T15.

Efeito positivo nas habilidades cognitivas. Os professores reconhecem que o conhecimento de vários idiomas proporciona benefícios no desenvolvimento cognitivo. habilidades ligadas à programação. Nesta categoria, relatamos duas subcategorias. Primeiro, os professores revelam que os alunos transferem habilidades de resolução de problemas e pensamento de nível superior do PL1 para o PL2. T7 explica que “os alunos aprendem que quando têm um problema precisam se dividir Divida-o em partes e, em seguida, eles podem resolver essas pequenas partes uma a uma. Ao trabalhar com várias linguagens de programação, os alunos perceberiam que... A forma de pensar permanece a mesma”. T19 acrescenta que a aprendizagem múltipla Aprender idiomas ajuda mais no processo de pensamento, no pensamento de nível superior. sobre o que as linguagens de programação fazem”. Em segundo lugar, alguns professores enfatizam Aprender vários idiomas amplia a perspectiva dos alunos.

sobre linguagens de programação e suas aplicações. Por exemplo, Segundo a T20, “Conhecer vários idiomas permite ao aluno para fazer coisas diferentes”, e os alunos então sabem que “uma linguagem de programação específica não é sagrada”, de acordo com T18.

4.2.2 Problemas observados na transição para outra linguagem de programação . Os professores, por outro lado, identificam problemas quando os alunos migram da LP1 para a LP2. Categorizamos esses problemas em quatro categorias principais: Efeito negativo da transferência sintática (8 professores), Efeito negativo da transferência semântica (10 professores), os alunos fazem Sem ligação com o idioma anterior (4 professores) e alunos. desmotivados/menos confiantes para aprender um novo idioma (3 professores).

Efeito negativo da transferência sintática. Os professores identificam a sintaxe como um grande problema. problema ao migrar para uma nova linguagem de programação. Temos Nesta categoria existem duas subcategorias. Uma subcategoria se relaciona a a mudança de contexto dos alunos de uma língua para outra, que é difícil e demorado, levando em consideração as diferenças de sintaxe.

As diferenças entre idiomas às vezes são grandes. A T1 acredita que os alunos “Tenho mais dificuldade com a sintaxe do que com os conceitos”, enquanto a T2 afirma que um novo

A linguagem “cria uma barreira com a sintaxe”. A segunda subcategoria concentra-se nos problemas que surgem quando os alunos misturam sintaxe.

“ elementos de duas linguagens. Por exemplo, T21 menciona que os alunos às vezes escrevem PHP no estilo Python: dois pontos em vez de, ou misturado com, parênteses para loops”.

Efeito negativo da transferência semântica. Esta categoria relaciona-se com o compreensão dos alunos sobre a semântica de símbolos específicos ou

construções em uma nova linguagem de programação. Ela possui duas subcategorias . Primeiro, os alunos têm concepções errôneas baseadas em conhecimentos prévios . Podem ser conhecimentos de outra área, como matemática, ou conhecimento adquirido de uma aprendizagem prévia de programação linguagem. Por exemplo, T1 diz que “O símbolo = é um problema, eles “transferir a matemática para igualar”, e T17 diz “por exemplo, com o Structurizr

Em Python, você precisava usar um 'e' em vez de um '+' para concatenar cordas . A segunda subcategoria relaciona-se a uma luta geral com semântica. Grande parte dessa luta decorre da transição entre Linguagens tipadas estritamente e menos estritamente. T10 destaca um exemplo: “Quando os alunos migram do Python para o Java, a passagem de parâmetros é um desafio porque o Python não pede explicitamente que você declare seu tipos de dados enquanto o Java faz”.

Os alunos não estabelecem nenhuma conexão com o idioma anterior. Esta categoria

de problemas aponta o caso da transferência que não teve sucesso como Os alunos não conseguem conectar sua compreensão do mesmo conceito entre diferentes idiomas. Observamos que as quatro ocorrências disso A categoria se refere à transição de alunos de um ambiente de programação baseado em blocos , como o Scratch, para outra linguagem de programação. Por exemplo, T8 descreve o problema: Acho que eles estão com problemas de transferência. para o Scratch, acho que é porque eles recebem um bloco no Scratch e

Eles não pensam no que há dentro dos blocos, então simplesmente os puxam para dentro.

ICER 2021, 16 a 19 de agosto de 2021, Evento Virtual, EUA

Ethel Tshukudu, Quintin Cutts, Olivier Goletti, Alaaeddin Swidan e Felienne Hermans

Eles não percebem que é um bloco de repetição, que é o mesmo que um loop "for" em Python.

Alunos desmotivados ou com pouca confiança para aprender um novo idioma. Segundo três professores, a motivação para aprender uma nova linguagem de programação pode ser afetada. Isso acontece quando os alunos generalizam sua falta de habilidade em programação com base na primeira experiência "ruim" com a linguagem. Por exemplo, o professor T23 acredita que "Quando os alunos começam a aprender uma segunda linguagem, mas não se sentem confiantes nos conceitos e estruturas presentes em ambas, isso pode diminuir sua autoconfiança". A segunda subcategoria se refere aos alunos que permanecem presos a uma única linguagem de programação. De acordo com o professor T19, isso ocorre porque eles dominam uma linguagem e não veem valor em aprender outra, ou porque menosprezam outras linguagens.

4.3 RQ3: Opiniões sobre o uso de estratégias de transferência Ao explorar as opiniões dos professores de informática sobre o uso de estratégias de transferência, descobrimos que existem três principais perspectivas. A Tabela 4 apresenta um resumo dessas perspectivas.

4.3.1 Não acreditam que precisam implementar estratégias de transferência. Os resultados revelam que um número considerável de professores (12) não acredita que precise implementar estratégias de transferência em sala de aula ao ensinar uma segunda ou subsequente linguagem de programação. A justificativa para essa crença é dividida em cinco subcategorias.

Implementar estratégias de transferência leva tempo. Quatro professores expressaram que leva muito tempo para implementar essas estratégias, visto que estão limitados pelo tempo de aula. Por exemplo, o professor T10 disse: "Leva muito tempo para ensinar visando a transferência e usar isso como uma oportunidade de aprendizado. Leva tempo para desvendar os conceitos que os alunos estão aprendendo, mas não há tempo para isso".

A implementação de estratégias de transferência confunde os alunos. Dois professores expressaram a opinião de que as estratégias de transferência nem sempre produzem resultados positivos. Esses professores acreditam que a implementação de estratégias de transferência em sala de aula pode confundir os alunos, pois pode se tornar complicada. Por exemplo, o professor 2 afirma: "Se eu introduzir a semântica mais profunda de uma língua, é muito cedo, muitos alunos se perdem, não queremos que fique muito complicado".

A implementação de estratégias de transferência desmotiva os alunos. Dois professores também acreditavam que as estratégias de transferência poderiam gerar resultados negativos, como desmotivar os alunos a aprender um novo idioma. Por exemplo, o professor T8 afirmou: "Não ensinamos para a transferência, não vamos tão longe. Estou sempre preocupado em desmotivá-los".

A transição de um idioma anterior para um novo idioma é difícil. Dois professores relataram que consideram desafiador adaptar um ambiente baseado em blocos para um ambiente baseado em texto. Por exemplo, o professor 1 afirmou: "É difícil converter um ambiente de jogo do Scratch para Python".

Presume-se que os alunos façam a transferência implicitamente, portanto não há necessidade de estratégias de transferência. O último motivo pelo qual os professores (2) não implementam estratégias de transferência é porque acreditam que os alunos fazem a transferência automaticamente, sem necessidade de ajuda. Por exemplo, T11 afirma: "Acredito que a transferência acontece de forma bastante natural para os alunos do Scratch para o VB".

4.3.2 Acreditam que as estratégias de transferência são importantes e que o conhecimento de mais de um idioma ajuda a promover a transferência. Nove professores acreditam que as estratégias de transferência são importantes e que, quando os alunos conhecem mais de um idioma, isso lhes dá a capacidade de transferir conhecimento conceitual de um idioma para o outro. Por exemplo, o professor 19 afirma que: "Existem analogias entre os idiomas, mas os alunos não as percebem por si mesmos; alguém precisa lhes dizer. Com mais conceitos e analogias, os alunos podem aprender um novo idioma mais rapidamente. Os alunos se tornam mais resilientes a novos materiais/produtos porque já viram algo semelhante, o que ajuda mais no processo de pensamento, no pensamento de nível superior em relação ao que as linguagens de programação fazem".

4.3.3 Não acredito que os alunos possuam uma compreensão sólida da primeira língua a ponto de transferirem conhecimentos conceituais para uma nova língua. Quatro professores destacaram que os alunos não têm uma compreensão sólida da primeira língua, o que resulta em praticamente nada para transferir para a nova língua.

Esses professores acreditam que muitos não se concentram o suficiente no desenvolvimento do conhecimento conceitual ao ensinar as primeiras linguagens de programação. Por exemplo, T23 disse: "Acho que o maior risco é não ensinar a primeira linguagem de forma abrangente, especialmente o Scratch, que às vezes é visto como uma 'introdução' em vez de uma ferramenta de aprendizado". T3 também compartilha dessa visão e afirma: "Assim, não ensinamos os conceitos da primeira linguagem, mas sim o ambiente do jogo".

4.4 RQ4: Tipos de estratégias de transferência Ao analisarmos as

estratégias específicas que os professores utilizam para ensinar visando a transferência, observamos que, entre os 23 professores entrevistados, apenas nove utilizam estratégias de transferência intencionalmente e um o faz involuntariamente. As estratégias de transferência mais relevantes foram agrupadas em três categorias diferentes e são apresentadas na Tabela 5.

4.4.1 Referindo-se à primeira linguagem de programação. A primeira categoria principal de estratégias refere-se à primeira linguagem de programação ao ensinar a segunda. Esta categoria contém as três estratégias seguintes:

Referindo-se explicitamente à primeira linguagem de programação. Seis professores fizeram referência à primeira linguagem de programação. Essa estratégia visa preencher a lacuna entre um conceito usado na primeira linguagem e sua reutilização na segunda. Por exemplo, o professor T20 afirma que, ao ensinar uma nova linguagem, ele "apenas repetiria o conceito brevemente, mas não em detalhes, na nova linguagem. Indicaria as diferenças e semelhanças com a linguagem anterior".

Comparação de código em diferentes linguagens de programação. Cinco professores relataram ilustrar um conceito mostrando duas versões do mesmo programa, uma na primeira linguagem de programação e outra na nova linguagem. Às vezes, os professores até mostram as duas versões simultaneamente ou, indo ainda mais longe, detalham explicitamente o processo de tradução de uma representação para outra (bloco para texto, texto para texto, fluxogramas para texto). Por exemplo, o professor 13 afirma: "Para a maioria dos conceitos que apresento ao longo do curso, eu os mapeio para os conceitos do Scratch para mostrar como funcionam em ambas as linguagens". O professor 15 disse que, por exemplo, "mostraria um programa em JavaScript no quadro branco e, junto com os alunos, o transformaria em PHP".

Categoria	Número de professores
Não acredito que eles precisem implementar estratégias de transferência.	12
-A implementação de estratégias de transferência confunde os alunos.	2
-Não há tempo suficiente para implementar estratégias de transferência.	4
-A transposição de um idioma anterior para um novo idioma é difícil.	2
-Suponha que os alunos se transfiram implicitamente	2
-A implementação de estratégias de transferência desmotiva os alunos.	2
Acredito que o conhecimento de mais de um idioma facilita a transferência de conhecimento.	9
Não presumo que os alunos tenham uma compreensão sólida da língua materna.	4
para que quaisquer benefícios sejam transferidos para um novo idioma.	

Tabela 4: Opiniões sobre estratégias de transferência

Categoria	Número de professores
Referindo-se ao PL1	
- Referindo-se explicitamente a PL1	6
- Comparando código em diferentes linguagens de programação 5	
- Discutindo semelhanças 4	
Favorecer a transição	
- Dar ênfase aos conceitos	8
- Utilizando representações visuais	4
- Utilizando atividades específicas	3
- Evitar construções específicas de cada idioma	1
Preparativos para a transferência	
- Referindo-se à vida cotidiana	3
- Reforçando o conhecimento inicial de PL	1

Tabela 5: Tipos de estratégias de transferência

Discutindo semelhanças. Quatro professores mencionaram que eles apontam identificar semelhanças e diferenças entre duas versões diferentes de uma resolução em duas linguagens de programação diferentes. Ao perguntar perguntas para um aluno como "Como você faria isso em C#/Unity?", para exemplo.

4.4.2 Favorecer a transição. Esta categoria consiste em quatro estratégias instrucionais que favorecem a transição de uma primeira programação para uma nova programação. idioma para um segundo.

Dando ênfase aos conceitos. Oito professores mencionaram dar ênfase a a ênfase em conceitos como estratégia de transferência. Por exemplo, nomear e abstrair conceitos ao vê-los pela primeira vez em a primeira linguagem de programação. Ela permitiu relembrar aos alunos de os conceitos vistos anteriormente na segunda língua. Às vezes, referindo-se explicitamente à primeira língua ou apenas mencionando a conceito. Por exemplo, T22 diz: "Ensinamos conceitos que temos ensinado anteriormente, mas destacamos como eles eram na língua materna. quando os introduzimos na segunda língua."

Utilizando representações visuais. Quatro professores usam um nível intermediário. representações como fluxogramas servem para abstrair o conceito e torná-lo mais claro. Torne-o mais genérico antes de reutilizá-lo em uma segunda linguagem de programação. Por exemplo, T19 disse: "Primeiro, ensino-lhes o fluxo lógico em fluxogramas."

E quando migrei para Python, continuei mostrando os fluxogramas em um só. gráficos."

Utilizando atividades específicas. Três professores relataram preferir a transição entre linguagens de programação por meio de métodos pedagógicos estratégias ou ferramentas específicas. Uma estratégia mencionada foi permitir

Os alunos devem resolver uma tarefa no idioma de sua escolha.

Para reduzir a carga inerente ao problema, o T21 oferece aos alunos mesma tarefa na segunda língua que na primeira, então

que eles podem se concentrar apenas na nova sintaxe. Nós nos reagrupamos neste categorizar o uso de ferramentas específicas, como ambientes de desenvolvimento que permitem aos alunos alternar entre blocos e texto, e vice-versa.

T11 faz isso. A última atividade que vale a pena mencionar é a proposta de T15. tarefas em que os alunos têm de misturar duas línguas diferentes, obrigando-os a prestar atenção às diferenças de sintaxe.

Evitar construções específicas de cada idioma. Por fim, o T20 afirma que eles "tentam" para evitar conceitos muito específicos de JavaScript" quando outro Uma sintaxe mais genérica estava disponível para que a "troca fosse mais fácil".

4.4.3 Preparando-se para a transferência. Uma última categoria, menor, mas interessante, são os esforços feitos pelos professores para antecipar a transferência. Em particular, No exemplo a seguir, argumentamos que eles reforçam os alunos noivado.

Referindo-se ao cotidiano. Três professores mencionaram que consultam sites conhecidos pelos alunos e analisam o código-fonte. T13

Compararam as propriedades dos objetos em uma classe orientada a objetos com os campos de um elemento no software que utilizaram anteriormente em aula.

Reforçando o conhecimento da primeira linguagem de programação. Um professor mencionou que a transferência de conhecimento só ocorre se houver uma sólida compreensão da primeira linguagem de programação. Portanto, T23 afirmou que "os conceitos são transferidos se os alunos tiverem uma compreensão sólida de sua primeira linguagem, e essa compreensão é mais forte se os alunos tiverem a oportunidade de discutir, debater e questionar os conceitos". Mesmo que eles próprios não reconheçam isso como uma estratégia.

5 DISCUSSÃO

5.1 Reflexões sobre as Questões de Pesquisa 5.1.1 RQ1:

Razões para o ensino de múltiplas linguagens. Há uma grande variação nas razões pelas quais os professores ensinam múltiplas linguagens de programação. Alguns sentem que precisam fazê-lo porque faz parte do currículo. A maioria deles tende a associar as razões ao aumento do envolvimento e entusiasmo dos alunos pela programação; eles atraem os alunos com a primeira linguagem simples antes de lhes impor o trabalho árduo da segunda. Essas razões parecem ser justificáveis, considerando que trabalhos anteriores enfatizaram as dificuldades que os iniciantes enfrentam na compreensão da sintaxe [1, 10, 38]. Esses tipos de professores não acreditam que a primeira linguagem deva sequer ensinar conceitos, portanto, eles se concentram na criação de jogos como parte de suas aulas.

As visões deficitárias são, na verdade, um problema preocupante; não que estejam necessariamente erradas, mas a maioria dos professores em nosso estudo não mencionou a motivação para ensinar várias linguagens por acreditarem que elas podem ajudar os alunos a desenvolver a compreensão conceitual. Alguns trabalhos anteriores propuseram que, se os alunos aprenderem uma segunda ou subsequente linguagem de programação de forma adequada, seu conhecimento conceitual se desenvolve e se reestrutura [42]. Além disso, os currículos de Computação em anos anteriores incluíam disciplinas como Linguagens de Programação Comparadas, que eram populares e tinham o objetivo comum de aprofundar a compreensão conceitual dos alunos [16].

5.1.2 RQ2: Problemas/benefícios de múltiplas linguagens. Os professores revelaram que seus alunos enfrentam problemas de transferência. A maioria desses problemas já foi abordada em pesquisas anteriores sobre transferência de linguagens de programação. Sem instrução explícita, os alunos podem sofrer efeitos negativos da transferência semântica de linguagens anteriores ao aprenderem construções em uma nova linguagem [42]. Outros problemas relatados ocorrem quando os alunos não conseguem conectar conceitos comuns entre as duas linguagens, o que geralmente acontece quando aprendem conceitos abstratos em uma nova linguagem [25, 30, 41]. Outros problemas, como dificuldades de sintaxe, foram relatados em trabalhos anteriores que investigaram a resolução de problemas por alunos quase iniciantes em novas linguagens [2, 33, 47].

Apesar dessas falhas relatadas na transferência, os professores também relataram alguns sucessos na transferência conceitual. Estes geralmente ocorrem quando os alunos aprendem construções que compartilham sintaxe e semântica semelhantes na língua anterior e na nova língua [42]. Outros aspectos positivos, como a transferência de habilidades de resolução de problemas, foram relatados em [33–35] e também são atribuídos às semelhanças linguísticas.

Esses resultados mostram que os mesmos problemas surgem nas escolas como já aconteciam antes. foram relatados casos em universidades, e isso nos dá um propósito claro ao tentar implementar estratégias de intervenção de transferência nas escolas.

Essas intervenções podem ter uma chance de sucesso muito maior se forem bem executadas, visto que os sistemas escolares compartilham currículos e os professores têm uma cultura de adotar materiais em vez de desenvolvê-los do zero.

5.1.3 RQ3: Opiniões sobre o uso de estratégias de transferência. As crenças expressas pelos professores ao responderem à RQ1 moldaram significativamente suas opiniões e abordagens no ensino de segundas línguas. Por exemplo, a maioria dos professores em nosso estudo não acredita que a implementação de estratégias de transferência em sala de aula seja uma prioridade no ensino de uma segunda língua. Eles relatam suas próprias percepções sobre os desafios que podem surgir ao implementar estratégias de transferência. Suas opiniões implicam, em sua maioria, que eles não têm muita fé no conhecimento conceitual que os alunos adquiriram na primeira língua. O MPLT [42], no entanto, mostrou que, na maioria dos casos, os alunos transferem automaticamente conhecimento semântico e conceitual de línguas anteriores com base em similaridades sintáticas. Se os professores não estiverem cientes desses mecanismos de transferência, isso pode impactar negativamente a aprendizagem da segunda língua, como mostrado nos resultados da RQ2 e confirmado em trabalhos anteriores [8, 43]. Os 39% dos professores, no entanto, acreditam na importância das estratégias de transferência e reconhecem que o conhecimento de múltiplas linguagens de programação pelos alunos pode ajudar a promover a transferência de conhecimento conceitual e aprofundar a compreensão conceitual. Como resultado, esses professores tentam implementar alguma forma de estratégias de transferência na sala de aula (RQ4).

5.1.4 RQ4: Tipos de estratégias de transferência. As estratégias de transferência relatadas no estudo incluem principalmente a referência à primeira linguagem de programação. Estratégias que enfatizam a transferência conceitual, mostrando diferentes versões do mesmo código na linguagem anterior e na nova linguagem, ou atribuindo a mesma tarefa em duas linguagens diferentes, estão em consonância com as técnicas de Hugging e Bridging [28]. Outras estratégias utilizam a ideia de mostrar um conceito em um contexto diferente [11]. Essas estratégias explícitas de transferência foram utilizadas em trabalhos anteriores e demonstraram ser bem-sucedidas em auxiliar os alunos na transferência para novas linguagens [8, 18, 23, 39, 47].

Um professor também mencionou explicitamente a importância de ensinar aos alunos uma compreensão sólida dos conceitos na primeira linguagem de programação, o que está alinhado com o reforço da aprendizagem inicial [17].

Embora esses sejam esforços louváveis, apenas 39% dos professores utilizam estratégias de transferência, e algumas dessas estratégias parecem implementadas de forma apressada e informal. Por exemplo, os professores mencionam expressões como "Repito o conceito brevemente, mas sem muita profundidade" e "Sinalizo os conceitos".

5.2 Implicações para o Ensino de Ciência da Computação

Os resultados desta pesquisa indicam que um bom número de estratégias de transferência dos professores refletiu uma gama de boas abordagens amplamente promovidas na literatura sobre transferência [28, 43]. No entanto, a maioria dos professores não acreditava que precisasse implementar estratégias de transferência em sala de aula. Com base nesses resultados, apresentamos recomendações específicas para professores, pesquisadores e elaboradores de currículo sobre áreas em que mais esforços devem ser feitos para apoiar os professores na adoção de estratégias de transferência em sala de aula.

5.2.1 Recomendações para professores. Recomendamos que os professores considerem o conhecimento prévio de linguagem de programação dos seus alunos e criem atividades em sala de aula que estimulem o seu nível inicial de conhecimento.

compreensão para ajudá-los a desenvolver seu conhecimento conceitual à medida que aprendem novas linguagens de programação. Além disso, os professores devem usar instrução explícita para ajudar seus alunos a corrigir sintaxe negativa e transferência semântica de linguagens anteriores [43]. Por fim, os professores devem ser fortemente encorajados a acrescentar aos seus motivos para ensinar várias linguagens o benefício de aprofundar a compreensão conceitual sobre os conceitos de linguagem de programação. Ao fazer isso, eles apoiarão o desenvolvimento de habilidades de pensamento computacional muito mais fortes em seus alunos.

5.2.2 Recomendações para pesquisadores. Se as estratégias de transferência forem usadas efetivamente por professores de informática, é necessária uma pedagogia cuidadosamente planejada. Portanto, isso apresenta uma oportunidade para pesquisadores trabalharem com professores experientes para desenvolver, implementar e avaliar uma pedagogia onde as intervenções de transferência abordam problemas específicos de transferência dos alunos (por exemplo, conceitos, sintaxe e semântica) apresentados neste estudo e em pesquisas sobre transferência [27, 33, 42, 47]. A análise das percepções e opiniões recolhidas junto dos investigadores é um passo importante para melhorar o ensino da segunda e das subsequentes línguas de programação. Para além do trabalho existente sobre intervenções de transferência [8, 18, 43], é também necessária investigação adicional para compreender como o ensino para a transferência pode aprofundar o conhecimento conceptual e melhorar a aprendizagem da programação.

5.2.3 Recomendações para criadores de currículos. Os criadores de currículos de Ciência da Computação devem se basear em pesquisas e utilizá-las como guia para elaborar material didático que vise promover a transferência de conhecimento à medida que os alunos transitam entre linguagens de programação. Acreditamos que o objetivo de qualquer currículo deve ser facilitar a transferência de conhecimento, a progressão na aprendizagem e o desenvolvimento da compreensão conceitual. Além disso, os professores devem ter a oportunidade de participar de um programa de desenvolvimento profissional sobre transferência de conhecimento que aborde o valor do ensino de múltiplas linguagens de programação para o desenvolvimento do conhecimento conceitual, os benefícios e desafios da aprendizagem de uma segunda língua e as técnicas de ensino para a transferência de conhecimento.

5.3 Ameaças à validade

O objetivo deste artigo é compreender as perspectivas dos professores sobre a transferência de conhecimentos e as estratégias de transferência. Este estudo apresenta algumas ameaças à validade. Em primeiro lugar, entrevistamos professores apenas em dois países europeus, o que dificulta a generalização dos nossos resultados, uma vez que a situação em outros países pode ser diferente. Em segundo lugar, concentramo-nos apenas na situação do ensino médio, enquanto muitas escolas atualmente também ensinam programação a alunos mais jovens. Assim , as questões específicas de transferência de conhecimentos vivenciadas por alunos que chegam ao ensino médio com experiência em programação estão fora do escopo deste artigo.

6. CONSIDERAÇÕES FINAIS

Este estudo visa compreender as perspectivas e estratégias que professores do ensino médio empregam ao ensinar uma segunda ou subsequente linguagem de programação. Para tanto, realizamos entrevistas com 23 professores em exercício. Nossos principais achados incluem o fato de que muitos professores inicialmente utilizam linguagens de programação simples, criando, assim, uma oportunidade para a aprendizagem por transferência (RQ1). Além disso, os professores reconhecem tanto os benefícios quanto os problemas da transferência da LP1 para a LP2. Os benefícios relatados relacionam-se principalmente ao papel que a LP1 pode desempenhar na preparação para a LP2. Os problemas relatados por

Os problemas apontados pelos professores incluem: problemas de sintaxe, dificuldades semânticas (especialmente concepções errôneas) e o fato de os alunos, por vezes, não conseguirem perceber a conexão entre as duas línguas (RQ2). Observamos que, em sua maioria, os professores não veem necessidade de implementar estratégias de transferência e não aproveitam a oportunidade que a transferência poderia proporcionar (RQ3). Por fim, constatamos que alguns professores utilizam estratégias de transferência bem conhecidas (RQ4). Isso indica que trabalhos futuros poderiam se basear nessa disposição e disseminar o uso de estratégias de transferência eficazes .

Nosso artigo abre diversas outras possibilidades para pesquisas futuras.

Por exemplo, estudos iniciais mostraram que os resultados de aprendizagem poderiam ser melhorados se os métodos de ensino fossem ajustados de acordo com as previsões do MPLT [41, 43]. No entanto, essas intervenções não foram exploradas no contexto do ensino médio e não foram utilizadas para estudar a transferência de línguas baseadas em blocos para línguas baseadas em texto. Experimentos aprofundados como [42] nesses contextos seriam um próximo passo valioso. Pesquisadores podem realizar trabalhos futuros sobre transferência com professores que já consideram a importância do desenvolvimento conceitual e que já utilizam estratégias de transferência.

Se isso levar a melhores resultados, as descobertas podem fundamentar programas mais amplos de desenvolvimento profissional de professores sobre transferência de conhecimento.

REFERÊNCIAS

[1] Amjad Altadmri e Neil Brown. 2015. 37 milhões de compilações: investigando erros de programação de iniciantes em dados de estudantes em larga escala. SIGCSE 2015 - Anais do 46º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (2015), 522–527. <https://doi.org/10.1145/2676723.2677258> [2] Michal Armoni, Orni Meerbaum-Salant e Mordechai Ben-Ari. 2015. Do zero à programação "real". ACM Transactions on Computing Education (TOCE) 14, 4 (2015), 1–15.

[3] Deborah J. Armstrong e Bill C. Hardgrave. 2007. Compreendendo a aprendizagem Mindshift: a transição para o desenvolvimento orientado a objetos. MIS Quarterly 31, 3 (2007), 453–474. <http://www.jstor.org/stable/25148803>

[4] Katherine Bielaczyc, Peter L. Pirolli e Ann L. Brown. 1995. Treinamento em estratégias de autoexplicação e autorregulação: investigando os efeitos das atividades de aquisição de conhecimento na resolução de problemas. Cognição e instrução 13, 2 (1995), 221–252.

[5] John B. Biggs e Kevin F. Collis. 1982. Avaliação da Qualidade da Aprendizagem: A Taxonomia SOLO (Estrutura do Resultado de Aprendizagem Observado). Academic Press.

[6] Neil CC Brown, Sue Sentance, Tom Crick e Simon Humphreys. 2014. Reinício: O ressurgimento da ciência da computação nas escolas do Reino Unido. ACM Transactions on Computing Education (TOCE) 14, 2 (2014), 1–22.

[7] Victoria Clarke, Virginia Braun e Nikki Hayfield. 2015. Análise temática. Psicologia qualitativa: um guia prático para métodos de pesquisa (2015), 222–248.

[8] Wanda Dann, Dennis Cosgrove, Don Slater, Dave Culyba e Steve Cooper. 2012. Transferência mediada: Alice 3 para Java. In Anais do 43º Simpósio Técnico da ACM sobre Educação em Ciência da Computação. 141–146.

[9] Stephen Davies, Jennifer A Polack-Wahl e Karen Anewalt. 2011. Um panorama das práticas atuais no ensino da sequência introdutória de programação. Em Anais do 42º simpósio técnico da ACM sobre educação em ciência da computação. 625–630.

[10] Paul Denny, Andrew Luxton-Reilly, Ewan Tempero e Jacob Hendrickx. 2011. Compreendendo a barreira sintática para iniciantes. Em Anais da 16ª conferência conjunta anual sobre Inovação e tecnologia no ensino de ciência da computação. ACM, 208–212. <https://doi.org/10.1145/1999747.1999807> [11] Mary L. Gick e Keith J. Holyoak. 1983. Indução de esquemas e transferência analógica. Psicologia cognitiva 15, 1 (1983), 1–38.

[12] Olivier Goletti, Kim Mens e Feliene Hermans. 2021. Experiências de tutores no uso de estratégias explícitas em um curso introdutório de programação baseado em aprendizagem por problemas . In Anais da Conferência ACM de 2021 sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITICSE '21). ACM Press, Evento Virtual, Alemanha. <https://doi.org/10.1145/3430665.3456348> (no prelo).

[13] Michael Homer e James Noble. 2014. Combinando visualizações em mosaico e textuais de código. Em Anais da Segunda Conferência de Trabalho do IEEE sobre Visualização de Software . IEEE Computer Society, 1–10. <https://doi.org/10.1109/VISOFT.2014.11> [14] Cruz Izu, Amali Weerasinghe e Cheryl Pope. 2016. Um estudo das habilidades de design de código em programadores iniciantes usando a taxonomia SOLO. In Anais da Conferência ACM de 2016 sobre Pesquisa Internacional em Educação em Computação. ACM, Melbourne VIC Austrália, 251–259. <https://doi.org/10.1145/2960310.2960324>

ICER 2021, 16 a 19 de agosto de 2021, Evento Virtual, EUA

[15] Nan Jiang. 2000. Representação e desenvolvimento lexical em uma segunda língua. *Linguística Aplicada* 21 (03 2000). <https://doi.org/10.1093/applin/21.1.47> [16] KN King. 1992. A evolução do curso de linguagens de programação. Em *Anais do vigésimo terceiro simpósio técnico SIGCSE sobre educação em ciência da computação*. 213–219.

[17] David Klahr e Sharon McCoy Carver. 1988. Objetivos cognitivos em um currículo de depuração LOGO: instrução, aprendizagem e transferência. *Psicologia Cognitiva* 20, 3 (julho de 1988), 362–404. [https://doi.org/10.1016/0010-0285\(88\)90004-7](https://doi.org/10.1016/0010-0285(88)90004-7) [18] Michael Kölling, Neil CC Brown e Amjad Altadmri. 2015. Edição baseada em frames: facilitando a transição de blocos para programação baseada em texto. Em *Anais do Workshop em Educação em Computação para o Ensino Fundamental e Médio*. ACM, 29–38.

[19] Divna Krpan, Saša Mladenović e Goran Zaharija. 2017. Transferência mediada de linguagem de programação visual para linguagem de programação de alto nível. Em 2017 40ª Convenção Internacional sobre Tecnologia da Informação e Comunicação, Eletrônica e Microeletrônica (MIPRO). IEEE, 800–805.

[20] David J Malan e Henry H Leitner. 2007. Scratch para cientistas de computação iniciantes. *Boletim ACM Sigcse* 39, 1 (2007), 223–227.

[21] Lauren E. Margulieux, Mark Guzdial e Richard Catrambone. 2012. Material instrucional com subobjetivos melhora o desempenho e a transferência na aprendizagem do desenvolvimento de aplicativos móveis. In *Anais da Nona Conferência Internacional Anual sobre Pesquisa em Educação Computacional (ICER '12)*. Association for Computing Machinery, Nova York, NY, EUA, 71–78. <https://doi.org/10.1145/2361276.2361291> [22] Cindy Marling e David Juedes. 2016. CS0 para alunos de Ciência da Computação na Universidade de Ohio. Em *Anais do 47º Simpósio Técnico da ACM sobre Educação em Ciência da Computação*. 138–143.

[23] Monika Mladenović, Žana Žanko e Andrina Granić. 2021. Transferência mediada: Do texto aos blocos e vice-versa. *Jornal Internacional de Interação Criança-Computador* (2021), 100279.

[24] IT Chan Mow. 2008. Problemas e dificuldades no ensino de programação de computadores para iniciantes. Em *Técnicas inovadoras em tecnologia de instrução, E-learning, E-avaliação e educação*.

[25] Dale Parsons e Patricia Haden. 2007. Programação por osmose: Transferência de conhecimento de ambientes de programação imperativa para visual. Em *Anais da Vigésima Conferência Anual da NACCCQ*. 209–215.

[26] Elizabeth Patitsas, Michelle Craig e Steve Easterbrook. 2013. Comparar e contrastar diferentes algoritmos leva a um aumento na aprendizagem dos alunos. In *Anais da Nona Conferência Internacional Anual da ACM sobre Pesquisa em Educação Computacional Internacional - ICER '13*. ACM Press, San Diego, San California, EUA, 145. <https://doi.org/10.1145/2493394.2493409> [27] David Perkins e Fay Martin. 1985. Conhecimento frágil e estratégias negligenciadas em Programadores Iniciantes. IR85-22. (1985).

[28] David N Perkins, Gavriel Salomon, et al. 1992. Transferência de aprendizagem. *Internacional enciclopédia de educação* 2 (1992), 6452–6457.

[29] Martinha Piteira e Carlos Costa. 2013. Aprendendo Programação de Computadores: Estudo das Dificuldades na Aprendizagem de Programação. Em *Anais da Conferência Internacional de Sistemas de Informação e Design de Comunicação de 2013 (ISDOC '13)*. Association for Computing Machinery, Nova York, NY, EUA, 75–80. <https://doi.org/10.1145/2503859.2503871> event-place: Lisboa, Portugal.

[30] Kris Powers, Stacey Ecott e Leanne M Hirshfield. 2007. Através do espelho: ensinando CS0 com Alice. Em *Anais do 38º simpósio técnico SIGCSE sobre educação em ciência da computação*. 213–217.

[31] Igor Moreno Santos, Matthias Hauswirth e Nathaniel Nystrom. 2019. Experiências na transição da programação funcional para a programação orientada a objetos. Em *Anais do Simpósio ACM SIGPLAN de 2019 sobre SPLASHE*. 36–40.

[32] Jean Scholtz. 1996. Adaptação de planos de programação na transferência entre linguagens de programação: uma abordagem de desenvolvimento. Em *Estudos empíricos de programadores : Sexto workshop*. Intellect Books, 233.

Ethel Tshukudu, Quintin Cutts, Olivier Goletti, Alaaeddin Swidan e Felienne Hermans

[33] Jean Scholtz e Susan Wiedenbeck. 1990. Aprendendo segunda e subsequentes linguagens de programação: um problema de transferência. *International Journal of Human- Computer Interaction* 2, 1 (1990), 51–72.

[34] Jean Scholtz e Susan Wiedenbeck. 1991. Aprendendo uma nova linguagem de programação: um modelo do processo de planejamento. Em *Anais da Vigésima Quarta Conferência Internacional Anual do Havaí sobre Ciências de Sistemas*, Vol. 2. IEEE, 3–12.

[35] Jean Scholtz e Susan Wiedenbeck. 1993. Usando linguagens de programação desconhecidas: os efeitos na experiência. *Interacting with Computers* 5, 1 (1993), 13–30.

[36] Linda Seiter. 2015. Usando o SOLO para classificar as respostas de programação de alunos do ensino fundamental. In *Anais do 46º Simpósio Técnico da ACM sobre Educação em Ciência da Computação*. ACM, Kansas City, Missouri, EUA, 540–545. <https://doi.org/10.1145/2676723.2677244> [37] Andreas Stefik e Susanna Siebert. 2013. Uma investigação empírica sobre a sintaxe de linguagens de programação. *Trans. Comput. Educ.* 13, 4 (nov. 2013), 19:1– 19:40. <https://doi.org/10.1145/2534973>

[38] Andreas Stefik e Susanna Siebert. 2013. Uma investigação empírica sobre a sintaxe da linguagem de programação. *ACM Transactions on Computing Education (TOCE)* 13, 4 (2013), 1–40.

[39] Nour Tabet, Huda Gedawy, Hanan Alshikhabokar e Saquib Razak. 2016. De Alice ao Python. Introduzindo a programação baseada em texto no ensino fundamental II. In *Anais da Conferência ACM de 2016 sobre Inovação e Tecnologia no Ensino de Ciência da Computação*. 124–129.

[40] EL Thorndike e Robert S. Woodworth. 1901. A influência da melhoria de uma função mental sobre a eficiência de outras funções(I). *Revisão psicológica* 8, 3 (1901), 247.

[41] Ethel Tshukudu e Quintin Cutts. 2020. Transferência semântica em linguagens de programação: estudo exploratório de novatos relativos. Em *Anais da Conferência ACM de 2020 sobre Inovação e Tecnologia no Ensino de Ciência da Computação*. 307–313.

[42] Ethel Tshukudu e Quintin Cutts. 2020. Compreendendo a transferência conceitual para alunos que aprendem novas linguagens de programação. Em *Anais da Conferência ACM de 2020 sobre Pesquisa Internacional em Educação em Computação*. 227–237.

[43] Ethel Tshukudu e Siri Annethe Moe Jensen. 2020. O papel da instrução explícita na aprendizagem de uma segunda linguagem de programação por estudantes. Em *Conferência de Pesquisa em Educação Computacional do Reino Unido e Irlanda*. 10–16.

[44] TUG 2016. Estrutura de Ciência da Computação K–12. Recuperado em 20 de janeiro de 2021 de <http://www.k12cs.org> [45]

Karen P Walker e Stephen R Schach. 1996. Obstáculos à aprendizagem de uma segunda linguagem de programação: um estudo empírico. *Educação em Ciência da Computação* 7, 1 (1996), 1–20.

[46] David Weintrop e Uri Wilensky. 2018. Como as modalidades baseadas em blocos, baseadas em texto e híbridas bloco/texto moldam as práticas de programação de iniciantes. *International Journal of Child-Computer Interaction* 17 (2018), 83–92. <https://doi.org/10.1016/j.ijcci.2018.04.005>

[47] David Weintrop e Uri Wilensky. 2019. Transição de ambientes introdutórios baseados em blocos e em texto para linguagens de programação profissionais em salas de aula de ciência da computação do ensino médio. *Computers & Education* 142 (2019), 103646.

[48] Susan Wiedenbeck. 1993. Uma análise de programadores novatos aprendendo uma segunda linguagem. Em *Estudos empíricos de programadores: Quinto Workshop: Artigos apresentados no Quinto Workshop sobre estudos empíricos de programadores*, 3 a 5 de dezembro de 1993, Palo Alto, CA. Intellect Books, 187.

[49] Ursula Wolz, Henry H Leitner, David J Malan e John Maloney. 2009. Começando do zero em CS 1. Em *Anais do 40º simpósio técnico da ACM sobre educação em ciência da computação*. 2–3.

[50] Quanfeng Wu e John R Anderson. 1990. Transferência de resolução de problemas entre linguagens de programação. Relatório técnico. CARNEGIE-MELLON UNIV PITTS- BURGH PA INTELIGÊNCIA ARTIFICIAL E PSICOLOGIA